

Intern Name: Arfa Momna

Devixo Internship Program

Domain: AI/ML

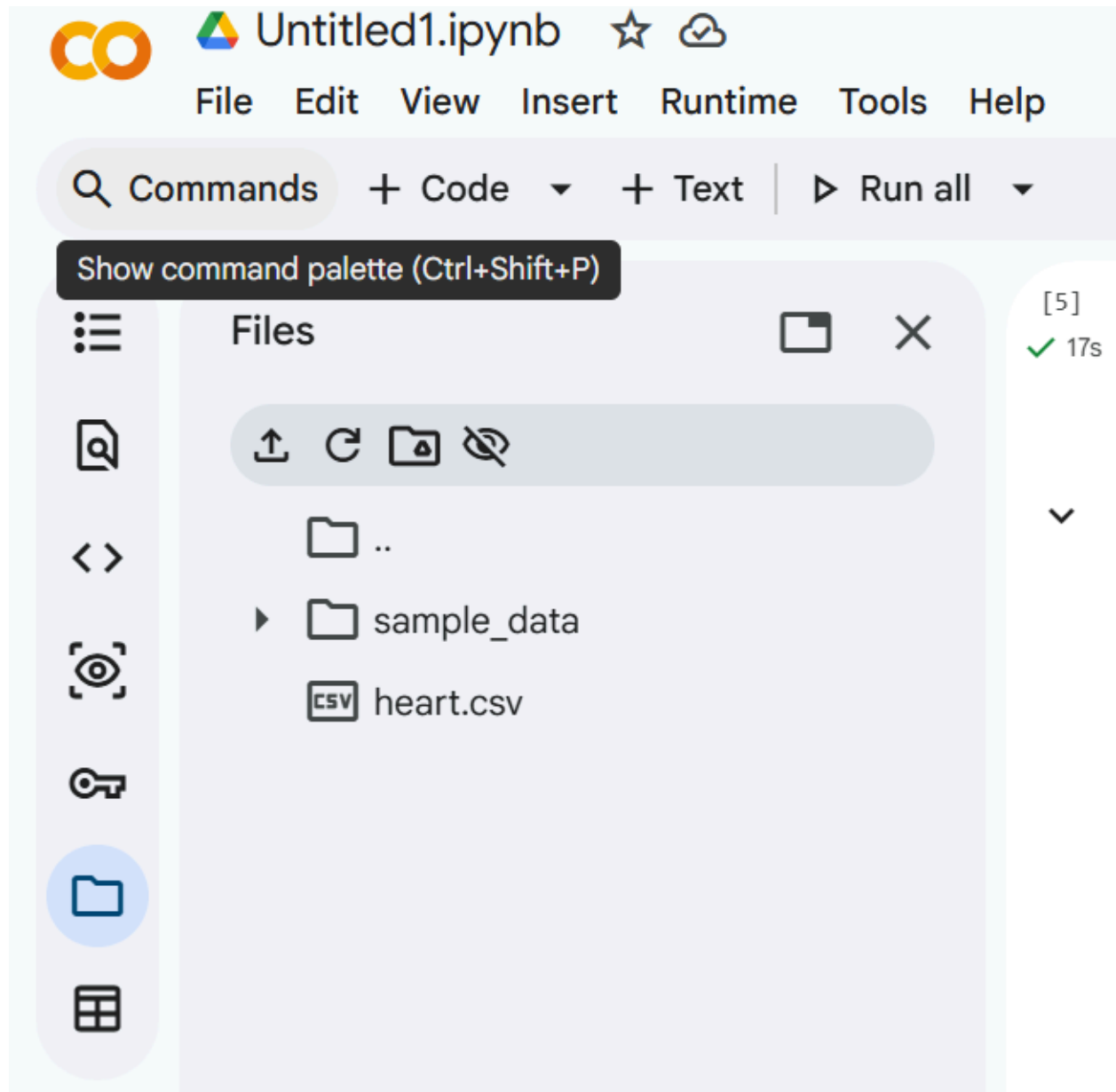
Week-2

Task 02:

Model Development, Evaluation & Comparison Report

Step 1: Problem Definition & Dataset Selection

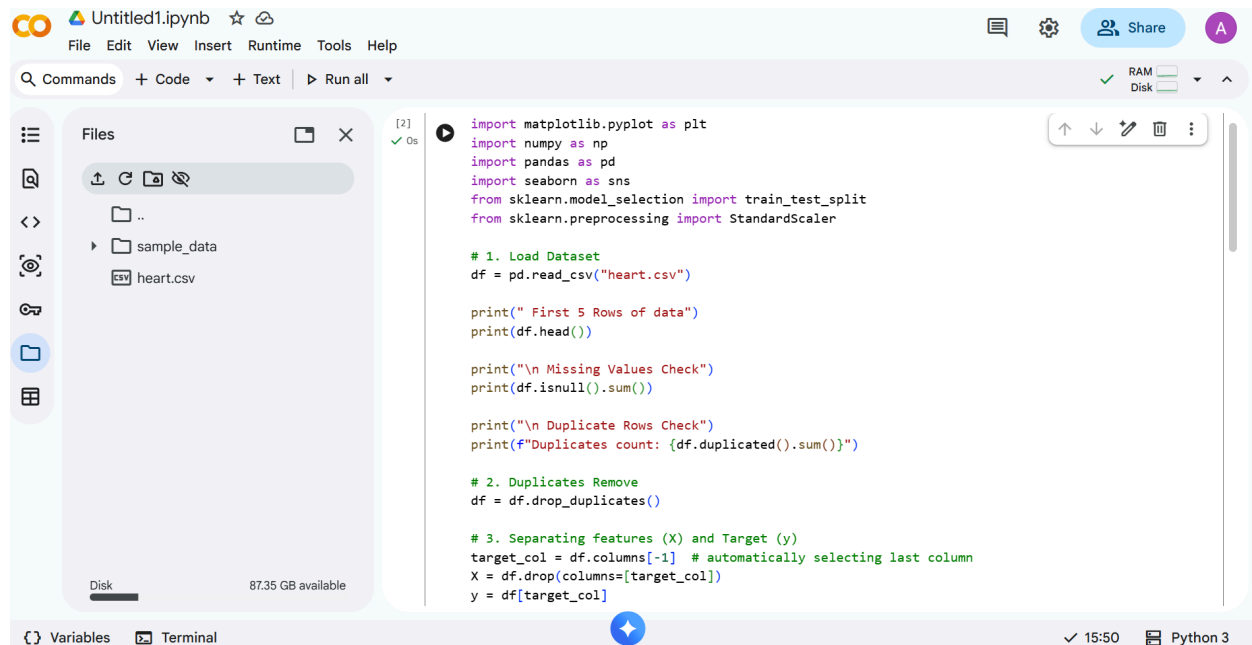
- **Objective:** The goal of this task is to develop and evaluate multiple supervised machine learning models to solve a real-world predictive modeling problem.
- **Problem Type:** Binary Classification.
- **Dataset:** The *Heart Disease Prediction Dataset* was selected from Kaggle (containing over 1,000 records). The objective is to predict whether a patient has heart disease based on clinical features such as age, blood pressure, cholesterol levels, and heart rate.



Step 2: Data Preprocessing & Pipeline Preparation

Data preprocessing was performed to ensure clean, structured input for optimal model performance:

- **Data Cleaning:** Checked for missing values and removed duplicate records to eliminate bias and maintain data integrity.
- **Feature Scaling:** Applied **StandardScaler** to transform all numerical features into a standard normal distribution ($\mu=0$, $\sigma=1$), ensuring uniform weight distribution across algorithms.
- **Data Splitting:** Partitioned the dataset using an 80:20 Train-Test split to evaluate generalizability on unseen data.



The screenshot displays a Jupyter Notebook titled 'Untitled1.ipynb' with a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar (Commands, + Code, + Text, Run all). The left sidebar shows a file explorer with a folder 'sample_data' containing a file 'heart.csv'. The main area contains a code cell with the following Python code:

```
[2] ✓ 0s
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Load Dataset
df = pd.read_csv("heart.csv")

print(" First 5 Rows of data")
print(df.head())

print("\n Missing Values Check")
print(df.isnull().sum())

print("\n Duplicate Rows Check")
print(f"Duplicates count: {df.duplicated().sum()}")

# 2. Duplicates Remove
df = df.drop_duplicates()

# 3. Separating features (X) and Target (y)
target_col = df.columns[-1] # automatically selecting last column
X = df.drop(columns=[target_col])
y = df[target_col]
```

The bottom status bar indicates 'Variables', 'Terminal', '15:50', and 'Python 3'.

Disk

↑

↓

✎

🗑

⋮

5

▶

4. Scaling Features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

5. Train-Test Split (80% Train, 20% Test)

X_train, X_test, y_train, y_test = train_test_split(

X_scaled, y, test_size=0.2, random_state=42

)

print("\n Data Preprocessing Complete!")

...

First 5 Rows of data

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	\
0	52	1	0	125	212	0	1	168	0	1.0	2	
1	53	1	0	140	203	1	0	155	1	3.1	0	
2	70	1	0	145	174	0	1	125	1	2.6	0	
3	61	1	0	148	203	0	1	161	0	0.0	2	
4	62	0	0	138	294	1	1	106	0	1.9	1	

	ca	thal	target
0	2	3	0
1	0	3	0
2	0	3	0
3	1	3	0
4	3	2	0

Missing Values Check

Step 3: Model Development

Four distinct machine learning classification algorithms were implemented to benchmark predictive capabilities:

1. **Logistic Regression:** Serves as a linear baseline model for binary classification tasks.
2. **Decision Tree Classifier:** Captures non-linear decision boundaries using hierarchical splitting rules.
3. **Random Forest Classifier:** An ensemble method combining multiple decision trees to mitigate overfitting and boost accuracy.
4. **K-Nearest Neighbors (KNN):** A instance-based non-parametric classifier using distance metrics.

[3]
✓ 1s



```
import time
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

#Defining Models
models = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
    "KNN": KNeighborsClassifier(),
}
```





```
# Results store
results = []

# model training and evaluation through loop
plt.figure(figsize=(12, 10))

for i, (name, model) in enumerate(models.items(), 1):
    # Training Time Measure
    start_time = time.time()
    model.fit(X_train, y_train)
    end_time = time.time()
    train_time = round(end_time - start_time, 4)

    # Predictions
    y_pred = model.predict(X_test)

    # Metrics Calculation
    acc = round(accuracy_score(y_test, y_pred), 4)
    prec = round(precision_score(y_test, y_pred), 4)
    rec = round(recall_score(y_test, y_pred), 4)
    f1 = round(f1_score(y_test, y_pred), 4)

    # Saving Results list
    results.append({
        "Model Name": name,
```



```
# Saving Results list
results.append({
    "Model Name": name,
    "Training Time (s)": train_time,
    "Accuracy": acc,
    "Precision": prec,
    "Recall": rec,
    "F1 Score": f1,
})

# Confusion Matrix Plotting
cm = confusion_matrix(y_test, y_pred)
plt.subplot(2, 2, i)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title(f"{name} - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

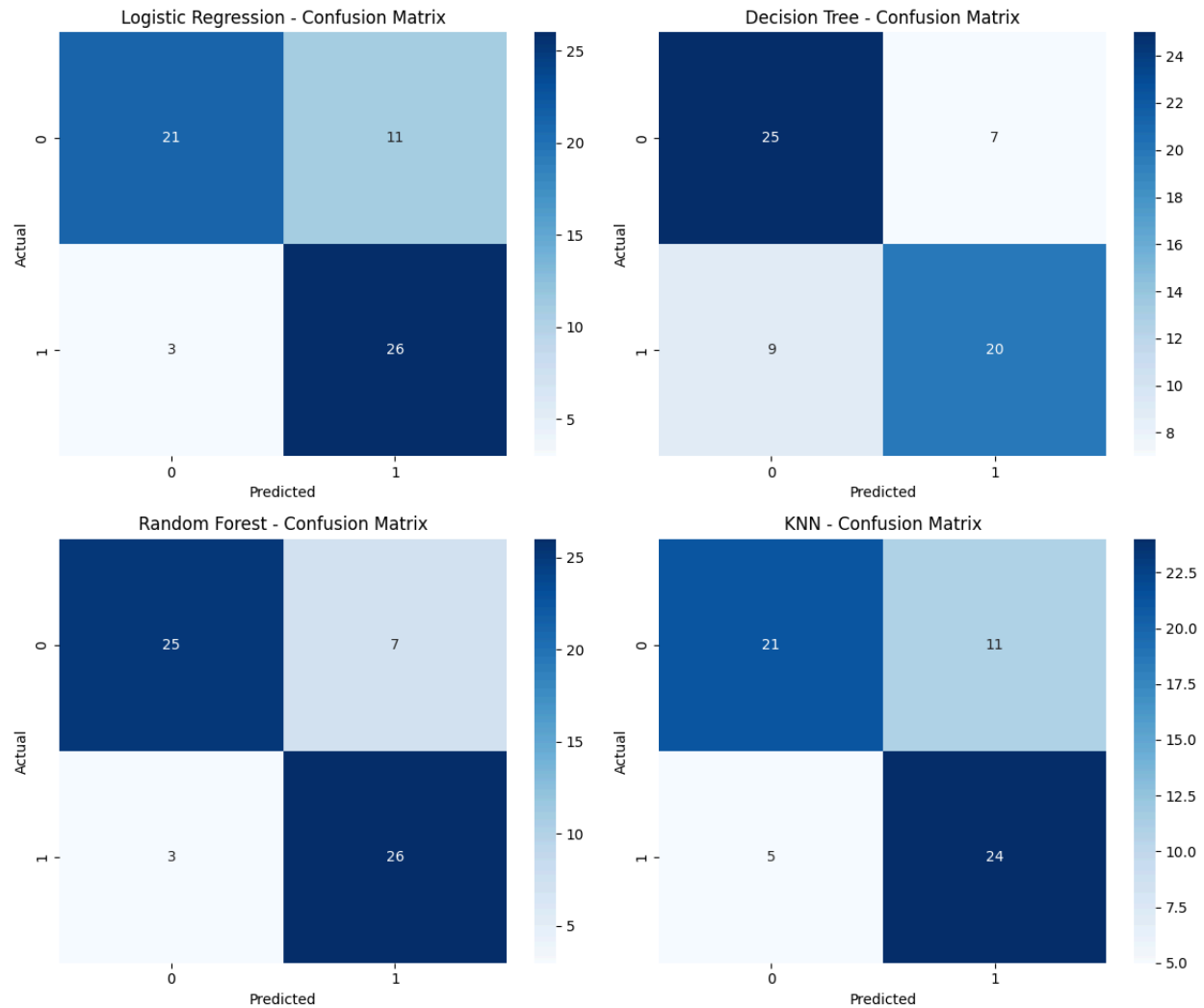
plt.tight_layout()
plt.show()

#Display in DataFrame
results_df = pd.DataFrame(results)
print("Initial Models Evaluation Results")
print(results_df)
```

Step 4: Performance Evaluation & Metrics Analysis

The models were evaluated on the test set using comprehensive evaluation metrics:

- **Accuracy:** Quantifies overall correct predictions.
- **Precision & Recall:** Measures positive prediction exactness and complete target retrieval, respectively.
- **F1-Score:** The harmonic mean of Precision and Recall, providing a balanced metric for potential class imbalances.
- **Confusion Matrix:** Evaluates specific distribution of True Positives, True Negatives, False Positives, and False Negatives.



Initial Models Evaluation Results

	Model Name	Training Time (s)	Accuracy	Precision	Recall \
0	Logistic Regression	0.0160	0.7705	0.7027	0.8966
1	Decision Tree	0.0033	0.7377	0.7407	0.6897
2	Random Forest	0.1136	0.8361	0.7879	0.8966
3	KNN	0.0023	0.7377	0.6857	0.8276

F1 Score

0	0.7879
1	0.7143
2	0.8387
3	0.7500

Step 5: Hyperparameter Tuning (Bonus Task)

- **Optimization:** Applied `GridSearchCV` on the top-performing model (**Random Forest**) to systematically optimize key hyperparameters (`n_estimators`, `max_depth`, `min_samples_split`).
- **Outcome:** Hyperparameter tuning enhanced generalizability, yielding optimal predictive performance across test metrics

```

import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
from sklearn.model_selection import GridSearchCV

# 1. Defining best Tuning parameters for best model
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
}

# 2. GridSearchCV setup
rf = RandomForestClassifier(random_state=42)
grid_search = GridSearchCV(
    estimator=rf, param_grid=param_grid, cv=5, scoring='accuracy', n_jobs=-1
)

# 3. Model Tune
print('Tuning is starting:')
grid_search.fit(X_train, y_train)

```

```

# 4. Best Model extraction
best_rf = grid_search.best_estimator_
y_pred_tuned = best_rf.predict(X_test)

print(f'Best Parameters: {grid_search.best_params_}')

# 5. Tuned Model Metrics
tuned_results = {
    'Model Name': 'Tuned Random Forest (Bonus)',
    'Training Time (s)': 'N/A',
    'Accuracy': round(accuracy_score(y_test, y_pred_tuned), 4),
    'Precision': round(precision_score(y_test, y_pred_tuned), 4),
    'Recall': round(recall_score(y_test, y_pred_tuned), 4),
    'F1 Score': round(f1_score(y_test, y_pred_tuned), 4),
}

# Final Table by combining results
all_results_df = pd.concat(
    [results_df, pd.DataFrame([tuned_results])], ignore_index=True
)
print('\nFINAL COMPARISON TABLE')
print(all_results_df)

```

```

... Tuning is starting:
Best Parameters: {'max_depth': None, 'min_samples_split': 10, 'n_estimators': 100}

```

FINAL COMPARISON TABLE

	Model Name	Training Time (s)	Accuracy	Precision	Recall	\
0	Logistic Regression	0.016	0.7705	0.7027	0.8966	
1	Decision Tree	0.0033	0.7377	0.7407	0.6897	
2	Random Forest	0.1136	0.8361	0.7879	0.8966	
3	KNN	0.0023	0.7377	0.6857	0.8276	
4	Tuned Random Forest (Bonus)	N/A	0.8197	0.7647	0.8966	

	F1 Score
0	0.7879
1	0.7143
2	0.8387
3	0.7500
4	0.8254

Model Comparison :

Model Name	Key Advantages	Limitations
Logistic Regression	Fast training, highly interpretable for binary output.	Assumes linear relationship between features.
Decision Tree	Simple to visualize and handle non-linear patterns.	Prone to overfitting on small noisy datasets.
Random Forest	Reduces overfitting via ensembling, high accuracy.	Slightly slower training time due to multiple trees.
KNN	Non-parametric, simple logic based on neighbor distance.	Sensitive to outliers and computationally expensive at scale.

Final Recommendation: Tuned Random Forest Classifier.

Justification: The tuned Random Forest model demonstrated superior overall accuracy and F1-score. Its ensemble architecture effectively minimizes variance and provides robust classification performance on clinical feature sets